RESUMEN SEGUNDO PARCIAL

TEMAS

- Filosofía Lean - Método Kanban en el contexto del desarrollo de software
- Métricas en los 3 enfoques (Tradicional / Lean / Agile)

Unidad 3: Gestión de software como producto

- Prácticas continuas

Unidad 4: Aseguramiento de Calidad de Proceso y de Producto

- Contenidos:
  - Conceptos generales sobre calidad.
  - Importancia de trabajar para y con Calidad. Ventajas y Desventajas.
  - Actividades relacionadas con el Aseguramiento de la Calidad del Software.
  - Principales Modelos de Calidad existentes (CMMI – SPICE – ISO).
  - Auditorías al Software: Auditorías de Proyecto y Auditorías al Grupo de Calidad.
  - Proceso de Auditorías
  - Calidad de Producto: Planificación de pruebas para el software- Niveles y tipos de pruebas para el software. Técnicas y herramientas para probar software.
  - Técnicas y Herramientas para la realización de revisiones técnicas del software.
  - Testing en ambientes Ágiles.

CHECKLIST

De arquitectura

- ¿Están identificados los componentes principales de la arquitectura y sus responsabilidades?

- ¿Están claramente definidas las interfaces y comunicaciones entre los componentes?
- ¿La arquitectura contempla todos los requerimientos funcionales del sistema?
- ¿Se consideran los requisitos no funcionales, como seguridad, rendimiento y disponibilidad?
- ¿Las dependencias entre componentes están justificadas y permiten realizar modificaciones sin afectar innecesariamente a otros módulos?
- ¿Existe consistencia entre los diagramas arquitectónicos y la documentación de requerimientos?
- ¿Se encuentran identificados y documentados los riesgos y las decisiones arquitectónicas importantes?

De casos de prueba

- ¿Cada caso de prueba está asociado a un requerimiento o criterio de aceptación
- ¿Están definidas las precondiciones necesarias para ejecutar cada caso de prueba?
- ¿Los pasos de ejecución están descritos de forma clara, ordenada y reproducible?
- ¿Están especificados los datos de entrada necesarios para ejecutar la prueba?
- ¿Los resultados esperados están claramente definidos y son verificables?
- ¿Se contemplan escenarios positivos, negativos y valores límite cuando corresponda?
- ¿Los casos de prueba cubren todos los criterios de aceptación correspondientes?

▼

Filosofía Lean - Método Kanban en el contexto del desarrollo de software

Lean es una filosofía de gestión que busca maximizar el valor entregado al cliente y minimizar los desperdicios.

Su objetivo no es simplemente trabajar más rápido, sino identificar qué actividades aportan valor y cuáles consumen recursos sin contribuir al resultado.

En desarrollo de software, Lean busca evitar construir funcionalidades innecesarias, reducir esperas, mejorar el flujo del trabajo y favorecer el aprendizaje.

4.2. Los siete principios Lean

La cátedra presenta siete principios fundamentales.

Principio	Explicación
1. Eliminar desperdicios	Identificar y eliminar actividades que no aportan valor.
2. Amplificar el aprendizaje	Aprender continuamente mediante feedback y experimentación.
3. Embeber la integridad conceptual	Mantener la coherencia y calidad del producto en su conjunto.
4. Diferir compromisos	Evitar tomar decisiones irreversibles cuando todavía existe demasiada incertidumbre.
5. Dar poder al equipo	Permitir que las personas que realizan el trabajo participen en las decisiones.
6. Ver el todo	Optimizar el sistema completo y no únicamente actividades individuales.
7. Entregar lo antes posible	Obtener valor y retroalimentación rápidamente.

DEAD VEE

Aspectos importantes de estos principios

Eliminar desperdicios: no significa eliminar cualquier actividad que no sea programación. Por ejemplo, una revisión de calidad puede ser necesaria porque previene errores y retrabajo.

Amplificar el aprendizaje: en software no siempre se conoce la mejor solución desde el comienzo. Los equipos necesitan experimentar y aprender del resultado.

Diferir compromisos: no significa postergar todas las decisiones, sino esperar hasta disponer de información suficiente cuando resulte conveniente hacerlo.

Ver el todo: optimizar solamente desarrollo no sirve si, por ejemplo, Testing se transforma en un cuello de botella y los cambios quedan esperando durante semanas.

Entregar lo antes posible: favorece validar que el producto realmente genera valor antes de continuar incorporando funcionalidades.

4.3. Tipos de desperdicio

La presentación identifica dos grandes categorías.

Tipo 1 — Desperdicio necesario: actividades que no generan valor directamente, pero que son necesarias bajo las condiciones actuales. El objetivo es minimizarlas.

Tipo 2 — Puro desperdicio: actividades que no aportan valor y no son necesarias. El objetivo es evitarlas.

Los 7+1 desperdicios Lean

La PPT compara los desperdicios de manufactura con sus equivalentes en servicios basados en conocimiento, como el desarrollo de software.

Desperdicio	En desarrollo de software
Inventario	Trabajo parcialmente terminado que todavía no genera valor.
Sobreproducción	Funcionalidades adicionales que el usuario no necesita.
Procesamiento extra	Actividades, pasos o procedimientos innecesarios.
Transporte	Transferencias de conocimiento y cambios de tareas.
Esperas	Demoras por dependencias, aprobaciones o disponibilidad de recursos.
Movimiento	Cambios de contexto y tiempo de preparación para cambiar de actividad.
Defectos	Errores que producen correcciones y retrabajo.

Desperdicio	En desarrollo de software
Talento no utilizado	Desaprovechar conocimientos y capacidades de las personas.

TEM SIP DT

El último se considera el desperdicio adicional, de allí la expresión 7+1 desperdicios.

Ejemplo: un desarrollador tiene cinco funcionalidades empezadas, dos esperando una revisión y tres bloqueadas por dependencias.

Aunque haya mucho trabajo iniciado, eso no significa que se esté entregando valor.

Lean intenta reducir ese inventario de trabajo y favorecer que las funcionalidades realmente se terminen.

Método Kanban en el desarrollo de software

Kanban es un método para definir, gestionar y mejorar servicios de trabajo del conocimiento. Se basa en visualizar el trabajo, limitar lo que se inicia y **gestionar el flujo** para entregar valor de forma predecible. Parte del proceso actual: no exige reemplazar de golpe los roles ni la organización.

Valores de Kanban

Los **nueve valores de Kanban** orientan el trabajo de las personas y la gestión del servicio:

Valor	Qué significa
Transparencia	Hacer visible el estado del trabajo, sus reglas y problemas.
Equilibrio	Balancear demanda y capacidad para evitar sobrecarga.
Colaboración	Trabajar juntos para resolver obstáculos y mejorar.
Foco en el cliente	Priorizar el valor recibido por quien solicita el trabajo.
Flujo	Hacer que las tareas avancen hasta completarse sin esperas innecesarias.
Liderazgo	Impulsar mejoras desde cualquier integrante o nivel.
Entendimiento	Conocer el proceso actual y sus problemas antes de cambiarlo.
Acuerdo	Introducir cambios con el compromiso de los involucrados.
Respeto	Considerar a las personas, roles y prácticas existentes.

Principios

Gestión del cambio (4):

1. **Comenzar con lo que se hace ahora:** observar el proceso real antes de intentar reemplazarlo.
2. **Respetar los roles y responsabilidades actuales:** no exigir reorganización inmediata.
3. **Acordar mejoras mediante cambios evolutivos:** introducir modificaciones pequeñas y graduales, evaluar resultados y seguir mejorando.
4. **Fomentar el liderazgo en todos los niveles:** cualquier integrante puede identificar problemas y proponer soluciones.

Entrega de servicios (3):

1. **Comprender las necesidades y expectativas del cliente:** conocer el valor esperado y el servicio que necesita.
2. **Gestionar el trabajo y permitir la autoorganización:** coordinar el flujo y facilitar la entrega de valor.
3. **Revisar periódicamente los servicios y sus políticas:** evaluar resultados y adaptar las reglas para mejorar el desempeño.

Las seis prácticas generales

Práctica	Propósito
Visualizar	Tablero con estados, tarjetas y bloqueos.
Limitar WIP	Restringir la cantidad de tareas simultáneas en progreso.
Gestionar el flujo	• Consiste en procurar que el trabajo avance por las diferentes etapas sin bloqueos ni acumulaciones innecesarias. • La solución no es necesariamente aumentar la velocidad de Desarrollo, sino mejorar el funcionamiento del proceso completo.
Hacer explícitas las políticas	• Las políticas definen las reglas para gestionar el trabajo • Deben ser acordadas por los involucrados, visibles, sencillas, claras, aplicables y fáciles de modificar.

Práctica	Propósito
Ciclos de retroalimentación	Revisar periódicamente cómo funciona el sistema.
Mejorar colaborativamente y experimentar	Introducir cambios graduales y medir sus resultados.

WIP (Work In Progress) es el **trabajo iniciado que aún no ha sido completado**. Limitar WIP significa establecer cuántas tareas pueden encontrarse simultáneamente en una etapa o en todo el sistema. Su objetivo es reducir multitarea, saturación, acumulación y tiempos de espera.

Ejemplo: si Desarrollo tiene un límite WIP de 3 y ya existen tres tareas activas, no se inicia una cuarta hasta liberar capacidad. La idea es **dejar de empezar y empezar a terminar**. Que todos estén ocupados no implica entregar valor más rápido.

Cuello de botella: si Desarrollo termina rápido pero Testing acumula trabajo, hay que mejorar el flujo completo, no solo programar más rápido.

Sistema Push vs. Pull

Push (empujar)	Pull (tirar)
Una etapa envía trabajo aunque la siguiente no tenga capacidad.	La etapa siguiente toma trabajo **cuando tiene capacidad disponible**.
Favorece acumulaciones.	Favorece controlar el WIP y finalizar tareas.

Ejemplo: Desarrollo termina cinco funcionalidades y Testing tiene capacidad para dos. En **Push** se envían las cinco aunque Testing esté saturado, acumulando tareas en espera. En **Pull**, Testing toma las tareas **cuando tiene capacidad**. De este modo se intenta controlar WIP y el flujo completo.

Clases de servicio

Clase	Característica
Expreso	Trabajo **urgente** que necesita atención excepcional. La PPT ejemplifica WIP = 1 para que las urgencias no saturen el sistema.
Fecha fija	Trabajo que debe completarse **antes de un vencimiento**; la prioridad aumenta si se acerca la fecha y peligra la entrega.

Clase	Característica
Estándar	Trabajo **habitual**; respeta capacidad, límites WIP y políticas normales, pudiendo usar FIFO.
Intangible	Trabajo cuyo **beneficio no es inmediato**, como mejoras internas o prevención de problemas futuros.

Son **políticas de ejemplo**, no reglas universales.

Cadencias

Reunión	Frecuencia ilustrativa	Para qué
Team Kanban Meeting	Diaria, por ejemplo	Revisar **flujo, bloqueos y tareas** que necesitan atención para finalizarse.
Replenishment Meeting	Semanal o a demanda	Seleccionar qué trabajo **ingresa al sistema cuando hay capacidad**, según prioridades.
Retrospective	Quincenal o mensual	Analizar resultados y acordar **acciones para mejorar el sistema**.

Para el parcial: Kanban **no es solamente un tablero**. Combina principios, visualización, límites WIP, sistema Pull, políticas, feedback y mejora evolutiva.

▼

Métricas en los 3 enfoques (Tradicional / Lean / Agile)

Una **métrica de software** es una medida cuantitativa de alguna propiedad de un **proceso**, **proyecto** o **producto** de software. Permite comprender lo que está ocurriendo, comparar resultados, detectar problemas y apoyar decisiones.

No se mide por medir: antes de utilizar una métrica hay que preguntarse qué información aporta, para quién resulta útil y qué decisión permite tomar. Según el enfoque (tradicional, ágil o Lean), se pone la atención en aspectos diferentes.

1. Enfoque tradicional

Tipo	Qué mide
Proceso	Evalúa **cómo funciona el proceso de desarrollo**, por ejemplo cuánto tarda una actividad o cuántos defectos se introducen y detectan.
Proyecto	Permite **controlar el avance del proyecto**, comparando esfuerzo, cronograma, costos, tareas y resultados frente a lo planificado.
Producto	Evalúa **características del software construido**, como tamaño, cobertura, defectos o atributos de calidad.

Cuatro medidas básicas: tamaño, esfuerzo, tiempo calendario y defectos. TDTE

- **Tamaño:** magnitud del producto o trabajo realizado. Puede expresarse, según el caso, en líneas de código, funcionalidades, requisitos o componentes.
- **Esfuerzo: cantidad de trabajo invertido**, generalmente en horas-persona. Suma el esfuerzo de todas las personas involucradas.
- **Tiempo calendario (duración): tiempo transcurrido** entre el inicio y final de una actividad o proyecto. No depende directamente del número de personas.
- **Defectos:** cantidad, tipo y gravedad de anomalías identificadas, útiles para conocer problemas del desarrollo o del producto.

Esfuerzo ≠ duración. Ejemplo: dos personas, 8 h/día durante 5 días → **80 horas-persona** de esfuerzo, pero **5 días laborales** de duración.

Las métricas pueden aplicarse a distintos **niveles de responsabilidad**:

Nivel	Qué se analiza	Ejemplos
Desarrollador	Actividades o componentes en los que participa.	Esfuerzo, duración de tareas, cobertura unitaria, defectos encontrados en revisiones.
Equipo de desarrollo	Avance y resultados del trabajo colectivo.	Hitos, tareas terminadas, requerimientos, pruebas, distribución del esfuerzo.
Organización	Desempeño agregado de proyectos y procesos.	Presupuesto planificado vs. real, duración, precisión de estimaciones, defectos tras un release.

Regla: las métricas se eligen por su utilidad para decidir y mejorar, no para acumular números ni comparar personas injustamente.

2. Enfoques ágiles

Velocidad (Velocity)

La **velocidad (Velocity)** mide la **cantidad de trabajo que un equipo logra completar** durante una iteración o Sprint, normalmente en **Story Points (SP)**. Se contabilizan únicamente historias terminadas según el criterio de Done, no aquellas empezadas o parcialmente avanzadas.

Se utiliza para planificar y prever entregas futuras **a partir de datos históricos del mismo equipo**. No representa tiempo directamente ni es una medida válida de productividad individual.

$$\text{Velocidad} = \sum \text{SP de historias terminadas}$$

Si se planificaron 40 SP pero solo se terminaron 25 SP, la velocidad del Sprint es **25 SP**. No se cuentan historias incompletas.

Se usa para prever entregas en un **mismo equipo**, no para comparar productividad individual entre equipos.

Capacidad (Capacity)

La **capacidad (Capacity)** representa el **tiempo o los recursos realmente disponibles** para que el equipo trabaje durante una iteración. Se calcula considerando integrantes, dedicación, feriados, vacaciones, ausencias y otras obligaciones. Puede expresarse en horas.

No debe confundirse con la velocidad: **capacidad = recursos/tiempo disponibles**; **velocidad = trabajo efectivamente terminado**.

$$\text{Capacidad} = \sum \text{Horas disponibles del equipo}$$

Ejemplo: tres integrantes disponibles 30 horas cada uno → **90 horas**. Considera feriados, ausencias, dedicación y otras tareas.

Velocidad ≠ capacidad: velocidad expresa trabajo completado (frecuentemente en SP); capacidad expresa tiempo disponible (frecuentemente en horas). **Story Points no son horas.**

RTF — Running Tested Features

RTF (Running Tested Features) indica cuántas **funcionalidades están implementadas, probadas y realmente funcionando** en el producto. Se concentra en el avance funcional verificado, no en tareas iniciadas, líneas de código o funcionalidades que todavía no pasaron las pruebas.

El número puede **aumentar** al incorporar funcionalidades nuevas que funcionan y **disminuir** cuando alguna existente deja de funcionar correctamente.

Ejemplo: había 2 funcionalidades operativas y se suman 3 más → RTF = **5**, siempre que las anteriores sigan funcionando.

3. Lean/Kanban: métricas de flujo

Métrica	Desde / hasta	Qué representa
Lead Time — tiempo de entrega	**Desde que el cliente solicita** un elemento hasta su **entrega**.	Mide el tiempo de entrega visto desde la **perspectiva del cliente**, incluyendo esperas previas y procesamiento.
Cycle Time — tiempo de ciclo	**Desde que comienza el trabajo** sobre el elemento hasta que finaliza.	Mide el tiempo transcurrido **dentro del proceso**; excluye la espera previa a empezar.
Touch Time — trabajo efectivo	Suma de los intervalos donde **efectivamente se trabaja** sobre el elemento.	Excluye esperas, bloqueos y tiempo en colas sin trabajo efectivo.

Ejemplo: solicitud día 1, comienzo día 4, entrega día 11, trabajo efectivo 4 días.

- **Lead Time = 10 días.**
- **Cycle Time = 7 días.**
- **Touch Time = 4 días.**

Habitualmente **Touch Time ≤ Cycle Time ≤ Lead Time**.

Eficiencia del ciclo de proceso

$$\text{Eficiencia} = \frac{\text{Touch Time}}{\text{Elapsed Time}} \times 100$$

La **eficiencia del ciclo de proceso** expresa qué proporción del tiempo transcurrido se destinó a **trabajo efectivo**, frente al tiempo perdido en esperas u otras actividades sin procesamiento.

Ejemplo: una funcionalidad tarda 10 días desde la solicitud hasta la entrega, pero solo recibe 4 días de trabajo efectivo. Si los 10 días se toman como **Elapsed Time**, entonces **eficiencia = 4 / 10 × 100 = 40 %**. El 60 % restante del intervalo no fue trabajo efectivo.

Atención en ejercicios: identificar explícitamente **qué intervalo se utiliza como Elapsed Time**. Una entrega puede tardar mucho no porque programar sea lento, sino porque permanece esperando o bloqueada.

4. Métricas orientadas al servicio

Concepto	Significado
Expectativa de nivel de servicio	Nivel de servicio que **el cliente espera recibir**, por ejemplo un plazo de respuesta.
Capacidad de nivel de servicio	Nivel de servicio que **el sistema realmente puede entregar** de acuerdo con su rendimiento y resultados históricos.
Acuerdo de nivel de servicio	**Compromiso establecido con el cliente** respecto de características o plazos del servicio.
Umbral de adecuación	**Nivel mínimo aceptable**: por debajo de él, el servicio deja de satisfacer al cliente.

E C A U

Ejemplo: si el cliente espera que una solicitud se resuelva en 5 días, el equipo debe comparar esa expectativa con sus tiempos reales antes de asumir un compromiso.

5. Métricas de producto

- **Tamaño del producto:** mide la dimensión del software, por ejemplo por líneas de código, clases, requerimientos, funcionalidades o casos de uso. El tamaño medido y los Story Points **no significan lo mismo**.
- **Cobertura:** indica qué proporción de elementos o comportamientos quedó alcanzada por las pruebas, según el criterio adoptado.
- **Defectos por severidad:** cantidad y distribución de problemas según su impacto, útil para priorizar correcciones y evaluar el estado del

producto.

- **Densidad de defectos:** cantidad de defectos respecto del **tamaño del producto**. Para interpretarla, debe explicitarse cómo se midió ese tamaño.

Para el parcial: distinguir **esfuerzo y duración**, **velocidad y capacidad**, **RTF**, calcular **Lead/Cycle/Touch Time y eficiencia** y explicar para qué se usan las métricas de cada enfoque.

▼

Unidad 3: Gestión del software como producto

Prácticas continuas

Las **prácticas continuas** buscan incorporar las actividades de desarrollo, integración, verificación y entrega **durante toda la evolución del producto**, evitando concentrarlas únicamente al final. Su propósito es obtener retroalimentación temprana, integrar cambios pequeños, detectar defectos cuanto antes y mantener el software en condiciones de evolución.

En las PPT aparecen especialmente **integración continua, testing concurrente, TDD, ATDD, pruebas automatizadas y control de versiones**. No existe una presentación independiente que desarrolle completamente CI/CD.

Integración continua (CI)

La **Integración Continua (CI)** consiste en incorporar con frecuencia los cambios realizados por distintos integrantes a una versión compartida del software y **verificar que continúan funcionando juntos**. Habitualmente se ejecutan compilaciones y pruebas automatizadas para detectar incompatibilidades rápidamente, antes de que se acumulen muchos cambios.

Ejemplo: dos desarrolladores modifican módulos diferentes. Al integrar sus cambios se ejecutan las pruebas y se detecta de inmediato si una modificación rompe la comunicación entre módulos.

Flujo: cambios → integración al repositorio → build → ejecución de pruebas → feedback.

Se apoya en **SCM**: repositorio, commits, ramas, merge y control de versiones.

Entrega continua vs. despliegue continuo

Práctica	Idea central
Integración continua (CI)	Integrar y verificar cambios frecuentemente.
Continuous Delivery	Mantener una versión verificada y lista para desplegar cuando se decida.
Continuous Deployment	Publicar automáticamente las versiones que cumplen las verificaciones.

Aclaración de fuentes: la diferencia entre Delivery y Deployment es una explicación complementaria; las PPT no la desarrollan formalmente.

TDD — Test Driven Development

TDD (Test Driven Development) es una técnica de **desarrollo guiado por pruebas** en la que se escribe primero un test que representa el comportamiento esperado y después el código de producción mínimo para satisfacerlo. No es simplemente «hacer más testing»: se usa el test para **guiar el diseño y la implementación**.

Sus dos prácticas centrales son **Test First Development** (escribir primero las pruebas, generalmente unitarias) y **Refactoring** (mejorar continuamente el código sin modificar su comportamiento).

El ciclo es:

1. **RED:** escribir un test que falle.
2. **GREEN:** escribir el código mínimo para que pase.
3. **REFACTOR:** mejorar la estructura sin cambiar el comportamiento externo.

Las tres leyes de TDD (Robert C. Martin):

1. **No escribir código de producción** sin haber escrito primero un test que falla.
2. **No escribir más código de prueba** del necesario para provocar el fallo.

3. **No escribir más código de producción** que el mínimo indispensable para hacer pasar ese test.

Secuencia práctica: escribir el test → lograr que compile → ejecutarlo y ver que falla (Red) → implementar lo mínimo → ejecutarlo y comprobar que pasa (Green) → refactorizar → volver a ejecutar las pruebas.

Beneficios: código más mantenible, robusto y predecible; facilita cambios, reduce duplicación y promueve componentes cohesivos y poco acoplados.

Ejemplo: escribir primero una prueba donde sumar(2, 3) = 5; comprobar que falla; implementar lo mínimo; refactorizar y volver a probar.

Refactoring

Refactoring es una transformación disciplinada que **mejora la estructura interna del software conservando su comportamiento externo observable**. Su objetivo es simplificar el diseño, eliminar duplicaciones y facilitar el mantenimiento, sin agregar nuevas funcionalidades.

Ejemplo: reemplazar varias porciones de código repetidas por una función común que siga produciendo los mismos resultados.

Cómo se realiza: primero comprobar que pasan las pruebas existentes; identificar la mejora; modificar el código; ejecutar nuevamente las pruebas para confirmar que el comportamiento no cambió. **No debe confundirse con agregar funcionalidades.**

ATDD — Acceptance Test Driven Development

ATDD (Acceptance Test Driven Development) consiste en definir **antes de implementar** las pruebas de aceptación que expresan el comportamiento esperado desde el punto de vista del negocio o del usuario. Favorece la colaboración entre quienes conocen las necesidades, quienes desarrollan y quienes prueban.

Ejemplo: antes de implementar una venta se acuerda que el sistema no debe permitir comprar una cantidad superior al stock disponible; esa regla se expresa como una prueba de aceptación y guía la implementación.

La diferencia central es que **TDD guía principalmente el código y su diseño**, mientras que **ATDD guía el comportamiento que el producto debe ofrecer al usuario**.

TDD	ATDD
Guía diseño e implementación del código.	Guía el comportamiento esperado por el cliente.
Principalmente pruebas unitarias.	Pruebas de aceptación, criterios de aceptación y reglas de negocio.
Enfoque técnico.	Colaboración entre negocio, desarrolladores y testers.

Testing continuo, automatización y regresión

Testing continuo significa que las pruebas forman parte del trabajo de desarrollo, en lugar de comenzar solo después de la implementación. Su objetivo es **detectar defectos temprano y obtener feedback rápido** sobre los cambios.

- **Automatización:** ejecutar frecuentemente pruebas de manera repetible y con menor esfuerzo manual.
- **Testing de regresión:** volver a probar después de un cambio para comprobar que las funcionalidades anteriores sigan funcionando.
- **Control de versiones:** mantener las pruebas junto con el código, de modo que cada versión incluya sus verificaciones correspondientes.

Relación: control de versiones → integración continua → pruebas frecuentes → software verificado → entregas más confiables.

Para el parcial: explicar **CI**, diferenciar **Delivery/Deployment**, conocer **Red–Green–Refactor**, las **tres leyes de TDD**, **refactoring** y **TDD vs. ATDD**.

▼

Unidad 4: Aseguramiento de Calidad de Proceso y de Producto

▼ Conceptos generales sobre calidad

La cátedra define la **calidad** como el conjunto de aspectos y características de un producto o servicio relacionados con su capacidad para **satisfacer necesidades manifiestas o implícitas**.

No significa simplemente que el sistema funcione: también debe resultar adecuado para los usuarios y cumplir las necesidades y expectativas correspondientes. La calidad es **relativa** a las personas, sus circunstancias, el contexto y el paso del tiempo. Un mismo software puede ser considerado adecuado por un usuario e insuficiente por otro.

Un software de calidad debe considerar al **cliente, usuario, gerencia, equipo de desarrollo y mantenimiento** y otros interesados. **Ejemplo:** un producto técnicamente correcto puede carecer de calidad desde la perspectiva de un usuario si es demasiado difícil de utilizar.

Cinco visiones de la calidad

Visión	Qué considera
Del usuario	La calidad se evalúa según el grado en que el producto **satisface las necesidades y expectativas** de quienes lo usan.
Del producto	Se analiza a partir de **características o atributos** observables del software, como confiabilidad y rendimiento.
De manufactura	La calidad se asocia con la **conformidad con especificaciones, estándares y procedimientos** de producción.
Basada en el valor	Considera la relación entre **calidad, utilidad y costo**; un producto puede evaluarse según lo que aporta respecto de lo que cuesta.
Trascendental	Considera la calidad como una **excelencia reconocible**, aunque difícil de expresar mediante una única definición o medida.

Calidad programada, realizada y necesaria

- **Calidad programada:** las características y condiciones de calidad que se **planificaron** para el producto.
- **Calidad realizada:** el nivel y las características de calidad que el equipo **efectivamente obtiene** al desarrollar el software.
- **Calidad necesaria:** el nivel de calidad que realmente se necesita para **satisfacer al usuario y al negocio**.

La gestión de calidad busca lograr la mayor coincidencia posible entre las tres. Si se construye algo que fue planificado pero que nadie

necesita, puede generarse **desperdicio**; si la calidad realizada no alcanza la necesaria, el resultado será **insatisfacción**. **Ejemplo:** desarrollar funcionalidades innecesarias puede cumplir el plan técnico sin aportar valor al usuario.

Calidad de proceso vs. calidad de producto

Calidad de proceso	Calidad de producto
Se concentra en **cómo se desarrolla** el software y en las actividades utilizadas.	Se concentra en **qué características presenta** el software obtenido.
Evalúa **métodos, procedimientos, prácticas y cumplimiento de estándares**.	Evalúa **funcionalidad, rendimiento, seguridad, fiabilidad, mantenibilidad** y otros atributos.
Busca **prevenir defectos** y mejorar la forma de trabajar.	Busca determinar si el **producto satisface los requisitos de calidad**.
Se relaciona con **auditorías, revisiones y modelos de procesos**.	Se relaciona con **testing, revisiones y modelos de calidad de producto**.

Un **proceso de software** es un conjunto estructurado de **actividades, métodos, prácticas y transformaciones** que las personas utilizan para desarrollar o mantener software y sus productos asociados. La PPT parte de la idea de proceso como una secuencia de pasos ejecutados para alcanzar un propósito.

Para definirlo deben establecerse: **quién** participa, **qué** debe hacer, **cuándo** lo hace y **cómo** se ejecuta cada actividad. Definir procesos favorece la coordinación, la repetibilidad y la mejora.

Idea clave: mejorar el proceso favorece la calidad del producto, aunque no la garantiza por sí solo.

▼

Importancia de trabajar para y con Calidad. Ventajas y Desventajas

3.4. Importancia de trabajar con calidad

La presentación señala problemas habituales en proyectos de software:

- Atrasos en las entregas.

- Costos excedidos.
- Incumplimiento de compromisos.
- Requerimientos poco claros.
- Software que no cumple lo esperado.
- Horas extra y retrabajo.
- Dificultades para localizar o mantener componentes.

Trabajar con calidad ayuda a prevenir estos problemas mediante procesos definidos, controles, evaluaciones y mejora continua.

Principios de calidad según la cátedra

La diapositiva 8 presenta los siguientes principios:

1. La calidad no se inyecta ni se compra: debe estar incorporada desde el comienzo.
2. Es un esfuerzo de todos los integrantes.
3. Las personas son fundamentales y deben capacitarse.
4. Se necesita apoyo gerencial.
5. Hay que liderar mediante el ejemplo.
6. No se puede controlar lo que no se mide.
7. Se debe comenzar por lo básico, con simplicidad.
8. El aseguramiento de calidad debe planificarse.
9. Aumentar las pruebas no significa automáticamente aumentar la calidad.
10. Las prácticas de calidad deben ser razonables para el negocio.

Ventajas de trabajar con calidad

Permite reducir defectos y retrabajo, mejorar la previsibilidad, satisfacer a los clientes y facilitar el mantenimiento del software.

También favorece el control de los procesos y la posibilidad de repetir buenas prácticas.

Desventajas - Costos y dificultades

Implementar prácticas de calidad requiere capacitación, tiempo, herramientas, planificación y participación de los integrantes.

Las revisiones y controles también consumen recursos.

La cátedra destaca que estos esfuerzos pueden aumentar inicialmente el trabajo, pero los beneficios de prevenir defectos y reducir el retrabajo superan esos costos.

Su conclusión es que **trabajar con calidad resulta más barato que desarrollar sin calidad y corregir después los problemas**.

▼

Actividades relacionadas con el Aseguramiento de la Calidad del Software

La **Administración de Calidad del Software** comprende las actividades destinadas a establecer los niveles de calidad esperados, definir procesos y estándares y verificar que se respeten. Busca que el producto satisfaga los requisitos de calidad del proyecto y del negocio.

La cátedra enfatiza construir una **cultura de calidad**: no es responsabilidad exclusiva de un grupo de pruebas o aseguramiento; todas las personas del proyecto participan en su construcción.

Aseguramiento, planificación y control

Actividad	Función
Aseguramiento de calidad (SQA)	Define a nivel organizacional **estándares, procedimientos y métodos** para desarrollar software con calidad y evaluar su cumplimiento.
Planificación de calidad	Identifica los **estándares y procedimientos aplicables a un proyecto**, los adapta a su contexto y establece cómo evaluar resultados.
Control de calidad	Comprueba que el desarrollo **siga los procedimientos y estándares establecidos** mediante revisiones, evaluaciones y mediciones.

Ejemplo: se exige revisar requerimientos. Aseguramiento define el procedimiento, planificación decide cómo aplicarlo en el proyecto y control comprueba que se realizó.

Funciones del SQA

Las funciones de **SQA** incluyen definir herramientas, técnicas, métodos y estándares;

evaluar la **planificación del proyecto, requerimientos, diseño, programación, integración y pruebas**;

evaluar las prácticas de seguimiento y control;

detectar y comunicar **desviaciones**;

y adaptar las prácticas de aseguramiento al contexto del proyecto.

El objetivo es **prevenir problemas y comprobar de manera objetiva** cómo se aplican los procesos, no simplemente ejecutar pruebas de producto.

Grupo de Aseguramiento de Calidad (GAC)

El **Grupo de Aseguramiento de Calidad (GAC)** es responsable de realizar actividades de QA. Debe mantener una **independencia suficiente** para evaluar los proyectos objetivamente, incluso cuando identifique incumplimientos.

La PPT indica que **no debería reportar directamente al Gerente de Proyectos**, que no debería haber más de un nivel intermedio entre el GAC y la gerencia superior y que, cuando sea posible, debería reportar a una autoridad realmente interesada en la calidad.

¿Por qué? Si el grupo dependiera de la misma persona cuya gestión evalúa, podría verse comprometida su objetividad y su capacidad de informar problemas.

Estándares y Plan de Calidad

- **Estándar de producto:** establece **características o condiciones que debe cumplir un producto**. Por ejemplo, convenciones de codificación o estructura de documentos.
- **Estándar de proceso:** establece **cómo deben ejecutarse determinadas actividades**. Por ejemplo, el procedimiento de revisión de requerimientos.
- **Pla**n de Ca**lidad:** documento que especifica los **productos y niveles de calidad deseados**, los atributos más importantes, los estándares aplicables, las adaptaciones necesarias y **cómo y cuándo se**

evaluará su cumplimiento. Ejemplo: para un sistema bancario puede priorizar seguridad, fiabilidad y disponibilidad, con pruebas y revisiones asociadas.

El **Control de Calidad** puede utilizar **revisiones** de procesos y productos, y **evaluaciones automáticas o mediciones** para comprobar conformidad.

No se aplican estándares inapropiados solo por estar definidos.

Ciclo de mejora

El ciclo de mejora se basa en la relación **Definir procesos → desarrollar producto → evaluar calidad**.

- Si la calidad **no es adecuada**, se identifican problemas, se **mejoran los procesos** y se vuelve a evaluar.
- Si la calidad **es adecuada**, las prácticas efectivas se **estandarizan** para que puedan repetirse.

La evaluación retroalimenta el desarrollo y permite sostener la **mejora continua**.

Para el parcial: distinguir **SQA, planificación y control**, funciones e **independencia del GAC**, estándar de producto/proceso y contenido del **Plan de Calidad**.

▼

Principales Modelos de Calidad existentes (CMMI – SPICE)

Los **modelos de calidad** proporcionan marcos de referencia para evaluar cómo trabaja una organización o qué características presenta un producto. En el temario de este apartado interesan principalmente los **modelos de mejora de procesos**, que permiten diagnosticar prácticas actuales, reconocer debilidades, priorizar cambios y avanzar hacia procesos más controlados y predecibles.

CMMI — Capability Maturity Model Integration

CMMI (Capability Maturity Model Integration) es un **modelo de referencia para evaluar y mejorar la capacidad y madurez de los procesos de una organización**. Busca que los resultados del desarrollo

dependan menos de esfuerzos individuales y más de procesos definidos, gestionados y mejorados continuamente.

Surge como evolución del modelo SW-CMM y se asocia históricamente al **Software Engineering Institute (SEI)**. En el enfoque de la PPT, CMMI **no es una norma ISO** y se habla de **evaluaciones** realizadas por profesionales habilitados, no de certificar una norma ISO.

Constelaciones de la PPT:

- **CMMI-DEV:** desarrollo de productos y servicios.
- **CMMI-SVC:** gestión y entrega de servicios.
- **CMMI-ACQ:** adquisición de productos y servicios.

Representaciones de CMMI

Por etapas	Continua
Evalúa la **madurez global de una organización** y propone una evolución ordenada de sus procesos.	Evalúa la **capacidad de áreas de proceso particulares**, sin exigir que todas tengan el mismo nivel.
Utiliza **cinco niveles de madurez, del 1 al 5**.	Utiliza **seis niveles de capacidad, del 0 al 5**.
Permite expresar una **clasificación organizacional** y facilita comparaciones.	Permite **seleccionar áreas específicas** y definir prioridades para mejorar.

Cinco niveles de madurez

Nivel	Nombre	Característica
1	**Inicial**	El trabajo es **reactivo e impredecible**; los procesos están poco controlados y el resultado depende mucho de las personas.
2	**Administrado**	Se **planifican, siguen y controlan procesos en cada proyecto**. Se gestionan requerimientos, configuración y aseguramiento de calidad (**PPQA**).
3	**Definido**	La organización dispone de **procesos estandarizados y documentados** que pueden adaptarse a cada proyecto.

Nivel	Nombre	Característica
4	**Cuantitativamente Administrado**	Los procesos se **miden, analizan y controlan con datos cuantitativos** para comprender su comportamiento.
5	**Optimizado**	Se trabaja en la **mejora continua**, se estudian las causas de los problemas y se introducen mejoras preventivas.

DACOI

A mayor madurez, se busca mayor control y previsibilidad de los procesos.

SPICE

SPICE (Software Process Improvement and Capability Evaluation) es un marco vinculado con la **evaluación de la capacidad de los procesos y la mejora del desarrollo de software**. Permite analizar procesos existentes e identificar oportunidades de mejora.

La PPT lo presenta dentro de los modelos de mejora de procesos, pero **no desarrolla detalladamente sus niveles** como sí hace con CMMI. Para el parcial es importante distinguir que ambos se centran en la **calidad del proceso**, no en la ejecución de pruebas de un software específico.

Modelo IDEAL

Cinco etapas de mejora de procesos:

1. **Initiating (Iniciar):** establecer la necesidad de mejorar, sus objetivos generales y el **apoyo de la gerencia**.
2. **Diagnosing (Diagnosticar):** evaluar la situación actual para identificar **fortalezas, debilidades y oportunidades**.
3. **Establishing (Establecer):** seleccionar prioridades, definir objetivos específicos y **planificar las acciones** de mejora.
4. **Acting (Actuar):** llevar a la práctica los cambios planificados y evaluar su implementación.
5. **Learning (Aprender):** analizar resultados, extraer lecciones y utilizar ese conocimiento para futuras mejoras.

▼

Auditorías al Software — Puntos clave para el parcial

Una auditoría de software es una evaluación objetiva e independiente de los productos o procesos de software, cuyo propósito es verificar que cumplan con los estándares, procedimientos, especificaciones y lineamientos establecidos.

1. Los tres tipos de auditoría

Tipo	Qué evalúa
Auditoría de Proyecto	Verifica el **cumplimiento del proceso de desarrollo**, según lo establecido en el PACS (Plan de Aseguramiento de Calidad de Software). Comprueba la consistencia entre requerimientos, diseño, código y actividades de verificación y validación.
Auditoría de Configuración Funcional	Compara el software construido con los requerimientos de la ERS. Verifica que se implementen **solo y completamente** las funcionalidades especificadas y que la matriz de rastreabilidad esté actualizada.
Auditoría de Configuración Física	Comprueba que el software tal como está construido sea **consistente con su documentación técnica y de soporte**.

Para diferenciarlas:

- **Proyecto:** ¿se siguió el proceso de desarrollo definido?
- **Funcional:** ¿el producto cumple con los requerimientos?
- **Física:** ¿lo construido coincide con su documentación técnica?

2. Roles de una auditoría

Rol	Responsabilidades
Gerente de SQA	Prepara el plan de auditorías, calcula costos, asigna recursos y es responsable de resolver las no conformidades.
Auditor	Acuerda fecha y alcance, reúne y analiza evidencia objetiva, ejecuta la auditoría, elabora el informe y hace seguimiento de las acciones.

Rol	Responsabilidades
Auditado	Participa, presenta evidencia, responde al informe, propone el plan de acción y comunica su cumplimiento.

3. Las cuatro etapas del proceso de auditoría

1. **Preparación y planificación:** acordar fecha, lugar, objetivo, alcance y participantes; preparar documentación y checklist.
2. **Ejecución:** realizar la reunión de apertura, efectuar preguntas, solicitar evidencias y completar el checklist.
3. **Análisis y reporte del resultado:** evaluar hallazgos, preparar el informe preliminar, realizar la reunión de cierre y entregar el informe final.
4. **Seguimiento:** el auditado presenta un plan de acción para las desviaciones; el auditor revisa y verifica su cumplimiento y cierra las desviaciones corregidas.

Importante: la auditoría no termina al detectar una desviación; incluye el seguimiento de las acciones correctivas.

4. Diferencia entre desviación y observación

Resultado	Significado	¿Requiere plan de acción?
Desviación	Incumplimiento de un proceso, estándar o requerimiento documentado que puede afectar la conformidad o generar incertidumbre sobre la calidad.	**Sí.**
Observación	Condición o práctica que convendría mejorar y que puede convertirse en una desviación futura.	**No**, según la clasificación de la PPT.

Para el parcial: distinguir **los tres tipos de auditoría**, **sus tres roles**, **las cuatro etapas del proceso** y **desviación vs. observación**.

▼

Calidad de Producto: Planificación, niveles, tipos, técnicas y herramientas de Testing — Puntos clave para el parcial

8. Testing de Software — Calidad de Producto

Fuentes: PPT 09 — Testing de Software y PPT 09 — Testing de Software Estrategias Caja Negra.

8.1. ¿Qué es el Testing?

El **Testing de Software** es un conjunto de actividades que permite evaluar productos de software, identificar diferencias entre el comportamiento obtenido y el esperado, determinar si cumplen sus requisitos y **detectar defectos**.

La cátedra lo describe como un proceso **destructivo**, porque quien prueba busca encontrar problemas y demostrar que algo puede funcionar incorrectamente, en lugar de limitarse a comprobar que todo anda bien.

Importante: las pruebas pueden mostrar la presencia de defectos, pero **no garantizar su ausencia ni asegurar por sí solas la calidad del software**. La calidad también depende de actividades ejecutadas durante todo el proceso de desarrollo.

8.2. Error, defecto y falla; severidad y prioridad

Concepto	Definición
Error	Equivocación humana que ocurre al interpretar, diseñar o implementar.
Defecto	Anomalía que queda incorporada en un producto de trabajo, como un requerimiento, diseño o código.
Falla	Comportamiento incorrecto del software que se manifiesta durante su ejecución.

Ejemplo: una persona interpreta incorrectamente una regla de descuento (**error**), programa una comparación equivocada (**defecto**) y el sistema aplica un descuento a quien no corresponde (**falla**).

No confundir la gravedad del defecto con la urgencia de corregirlo:

Criterio	Qué representa	Clasificación de la PPT
Severidad	**Qué tan grave** es el problema y cuánto afecta al sistema.	Bloqueante, crítico, mayor, menor, cosmético.
Prioridad	**Qué tan pronto** debe solucionarse.	Urgencia, alta, media, baja.

Un defecto cosmético puede tener prioridad alta si afecta una demostración importante; y un defecto grave puede tener menor prioridad si la función involucrada no se utiliza todavía.

8.3. Los cuatro niveles de Testing

Nivel	¿Qué se prueba?	Ejemplo
Unitario	Una unidad o componente por separado, generalmente ejecutado por desarrolladores.	Calcular el importe de una compra.
Integración	La interacción e interfaces entre componentes o módulos.	Registrar una venta y actualizar el inventario.
Sistema	El sistema completo, incluidas sus funciones y atributos no funcionales.	Realizar una compra de principio a fin.
Aceptación	Que el producto satisface los requisitos y las necesidades del usuario o cliente.	El usuario valida que puede completar una compra.

Integración Top-down: se comienza desde módulos de nivel superior y se incorporan los inferiores progresivamente. **Bottom-up:** se empieza desde componentes inferiores hacia los superiores.

En las pruebas de aceptación se distinguen **Alfa** (entorno controlado) y **Beta** (uso en condiciones más reales, con usuarios representativos).

Para el parcial: distinguir el objetivo de cada nivel y poder identificarlo en un ejemplo.

8.4. Planificación y proceso de Testing

La cátedra organiza el trabajo de prueba en **cuatro actividades**:

1. **Planificación y control:** determinar qué se probará, objetivos, riesgos, estrategia, recursos, criterios de entrada y salida y seguimiento. Se documenta en un **Plan de Testing**.

2. **Análisis y diseño / especificación:** estudiar requerimientos, identificar condiciones de prueba, preparar casos, datos y ambientes.
3. **Ejecución:** ejecutar los casos, registrar resultados reales, compararlos con los esperados e informar defectos.
4. **Evaluación y reportes:** evaluar cumplimiento de objetivos, resultados y criterios de finalización y comunicar conclusiones.

Las pruebas deben planificarse **desde las etapas tempranas** del desarrollo; ejecutarlas únicamente al final dificulta detectar defectos antes de que se propaguen.

8.5. Casos de prueba

Un **caso de prueba** describe una situación concreta que se ejecutará para verificar un comportamiento y comparar un resultado real con uno esperado.

Debe indicar como mínimo:

- **Objetivo:** qué condición o requisito se intenta verificar.
- **Datos de entrada y ambiente:** valores, estado previo y condiciones necesarias.
- **Resultado esperado:** qué comportamiento debe producirse.

Ejemplo — iniciar sesión: objetivo: comprobar acceso válido; dato: usuario activo y contraseña correcta; resultado esperado: el sistema permite ingresar y muestra la pantalla correspondiente.

Un mismo criterio o condición de aceptación puede necesitar **más de una prueba**, por ejemplo un caso válido, uno inválido y valores límite.

8.6. Testing de regresión y Smoke Test

Regresión: volver a ejecutar casos de prueba después de modificar el software para comprobar que una corrección o funcionalidad nueva **no deterioró comportamientos que ya funcionaban**.

Smoke Test: conjunto breve de pruebas iniciales para comprobar que una versión tiene sus funciones básicas operativas y es **lo suficientemente estable como para continuar** con pruebas más detalladas.

Ejemplo: tras cambiar el cálculo de importes de una compra, se vuelve a comprobar ese cálculo y también el total y el comprobante (regresión). Antes de una ronda completa, se confirma que el sistema inicia y permite entrar (smoke).

8.7. Caja Negra vs. Caja Blanca

Caja Negra	Caja Blanca
Pruebas diseñadas desde **requisitos, entradas y salidas**.	Pruebas diseñadas teniendo en cuenta **la estructura interna del código**.
No exige conocer la implementación.	Requiere conocer decisiones, caminos o condiciones internas.
Técnicas: **partición de equivalencias y valores límite**.	Técnicas: **cobertura de sentencias, ramas o caminos**.

Son técnicas **complementarias**; probar caminos de código no asegura que se implementó lo que necesita el usuario, y probar solo salidas no garantiza cobertura interna.

8.8. Partición de equivalencias

Técnica de **Caja Negra** que divide el dominio de entrada en **clases válidas e inválidas** para las que se espera un comportamiento similar. Se elige al menos un representante de cada clase para reducir la cantidad de pruebas sin omitir categorías relevantes.

Ejemplo: cantidad permitida entre **1 y 100**, inclusive.

Clase	Valores incluidos	Tipo	Valor representativo
CE1	Menores que 1	Inválida	**0**
CE2	Entre 1 y 100	Válida	**50**
CE3	Mayores que 100	Inválida	**101**

El objetivo no es probar todos los números, sino probar **cada clase de comportamiento**.

8.9. Análisis de valores límite

También es de **Caja Negra**. Se concentra en los **extremos y valores inmediatamente adyacentes** a las clases porque allí son frecuentes los errores de comparación o interpretación.

Ejemplo: rango permitido 1–100. Probar **0, 1, 100 y 101**: justo fuera y justo dentro de ambos límites.

Diferencia para ejercicios:

- **Partición de equivalencias:** identificar clases y seleccionar representantes.
- **Valores límite:** seleccionar extremos y valores vecinos.

Si el rango excluye un extremo o tiene restricciones adicionales, los casos deben adaptarse a esas condiciones.

8.10. Técnicas basadas en la experiencia

- **Adivinanza de defectos:** el tester anticipa errores posibles basándose en experiencia y problemas habituales. Ejemplo: campos vacíos, datos duplicados o caracteres especiales.
- **Testing exploratorio:** el tester **aprende, diseña y ejecuta pruebas simultáneamente** mientras explora el sistema. No es probar de forma aleatoria ni sin propósito.

Estas prácticas complementan las técnicas sistemáticas.

8.11. Los siete principios del Testing

1. **El testing muestra la presencia de defectos, no su ausencia.**
2. **Testing exhaustivo es imposible:** las combinaciones posibles suelen ser demasiado numerosas.
3. **Testing temprano:** encontrar un problema antes reduce costos y retrabajo.
4. **Agrupamiento de defectos:** muchos problemas suelen concentrarse en pocos módulos.
5. **Paradoja del pesticida:** ejecutar siempre los mismos casos termina reduciendo la capacidad de hallar defectos nuevos; es necesario revisarlos y ampliarlos.
6. **El testing depende del contexto:** no se prueba igual un sistema bancario que un juego.
7. **Falacia de ausencia de errores:** un producto con pocos defectos detectados puede ser inútil si no satisface las necesidades del

usuario.

¿Cuánto testing es suficiente? No existe una cantidad universal; depende del contexto, riesgos, costo, tiempo y criterios de finalización.

8.12. Tipos de pruebas

Tipo	Qué se evalúa	Ejemplo
Funcionales	**Qué hace** el sistema; cumplimiento de reglas y operaciones.	Registrar un usuario, generar una factura.
No funcionales	**Cómo se comporta**; características de calidad.	Rendimiento, carga, estrés, volumen y seguridad.

Pruebas no funcionales importantes:

- **Performance (rendimiento):** tiempos de respuesta y rendimiento del sistema.
- **Carga:** comportamiento con determinada cantidad de usuarios, solicitudes o trabajo.
- **Estrés:** comportamiento ante condiciones extremas o sobrecarga.
- **Volumen:** comportamiento ante grandes cantidades de datos.
- **Seguridad:** accesos, permisos y protección de información.
- **Configuración e instalación:** funcionamiento en ambientes y configuraciones previstas.

Para el parcial: reconocer los **cuatro niveles**, proceso y elementos de los **casos de prueba**, **Caja Negra vs. Blanca**, resolver **clases de equivalencia y valores límite**, recordar **los siete principios** y distinguir **pruebas funcionales y no funcionales**.

▼

Técnicas y Herramientas para la realización de revisiones técnicas del software — Puntos clave para el parcial

7. Revisiones Técnicas del Software

1. ¿Qué son y para qué sirven?

Las **revisiones técnicas** son actividades de **Verificación y Validación (V&V) estáticas**: permiten identificar defectos en los productos de trabajo **sin ejecutar el software**.

Su objetivo principal es **detectar defectos lo más temprano posible**, antes de que se propaguen y generen retrabajo. Se pueden aplicar sobre requerimientos, diseños, código, planes de prueba y otros documentos o productos legibles del desarrollo.

Verificación: ¿estamos construyendo el producto correctamente, de acuerdo con sus especificaciones?

Validación: ¿estamos construyendo el producto correcto, de acuerdo con las necesidades del usuario?

2. Walkthrough vs. Inspección

Walkthrough (Recorrida)	Inspección
Informal; la dirige generalmente el autor del producto.	**Formal y sistemática**; la coordina un moderador.
Poca o ninguna preparación previa.	Requiere preparación individual anticipada.
Favorece el aprendizaje y el feedback rápido.	Busca identificar y remover defectos de manera eficiente.
No utiliza métricas formales ni un proceso controlado.	Utiliza **roles, checklists, registros y mediciones**.

Ejemplo de Walkthrough: un desarrollador presenta su diseño al equipo y recibe preguntas y comentarios sobre posibles problemas.

Ejemplo de Inspección: los inspectores revisan una especificación antes de una reunión formal, registran defectos y luego se verifica su corrección.

3. Los cinco roles de una Inspección

Rol	Responsabilidad
Autor	Crea o mantiene el producto inspeccionado; corrige los defectos e informa el retrabajo.
Moderador	Planifica, organiza y dirige la revisión; distribuye el material y hace seguimiento de los defectos.

Rol	Responsabilidad
Lector	Lee o presenta el producto durante la reunión.
Anotador	Registra los hallazgos de la revisión.
Inspector	Examina previamente el producto y detecta posibles defectos.

La PPT indica que el moderador debe proporcionar el producto a los inspectores **48 horas antes de la reunión**.

4. Las seis etapas del proceso de Inspección

- Duración máxima de la reunión: 2 horas.
- Anticipación para entregar el material: 48 horas.
- Responsable de organizar y coordinar la reunión: moderador.

1. **Planificación:** seleccionar el equipo, preparar el material y establecer el calendario.
2. **Visión general:** presentar el producto y el proceso de inspección.
3. **Preparación:** cada inspector analiza individualmente el producto para detectar defectos.
4. **Reunión de inspección:** compartir y registrar hallazgos, analizar posibles defectos y descartar falsos positivos.
5. **Corrección:** el autor corrige los defectos identificados.
6. **Seguimiento:** verificar que las correcciones estén realizadas y recolectar datos.

La reunión puede concluir con la **aceptación sin modificaciones**, el **rechazo** o la **aceptación provisional** del producto.

5. Reglas fundamentales

- **Revisar el producto, no al productor.** No se evalúa el desempeño de las personas.
- Establecer una agenda y respetarla; la PPT recomienda **reuniones de inspección de dos horas como máximo**.
- Registrar los defectos por escrito y utilizar **checklists**.

- Preparar anticipadamente el material y limitar el número de participantes.
- **Identificar los problemas, no intentar resolver todos durante la reunión.**
- Realizar seguimiento para comprobar que los defectos fueron corregidos.

Para el parcial: saber explicar el **propósito de las revisiones técnicas**, diferenciar **Walkthrough e Inspección**, y reconocer **los cinco roles y las seis etapas del proceso de Inspección**.

▼

Testing en ambientes Ágiles — Puntos clave para el parcial

9.1. ¿Qué es el Testing Ágil?

El **Testing Ágil** aplica los principios ágiles a las actividades de prueba. En lugar de esperar a que el software esté terminado, **integra las pruebas durante todo el desarrollo**. Busca recibir **feedback temprano**, **prevenir defectos**, automatizar verificaciones frecuentes y entregar software que funcione. La calidad pasa a ser una **responsabilidad compartida por todo el equipo**, aunque los testers aportan habilidades especializadas.

Enfoque tradicional vs. ágil: el tradicional suele concentrar pruebas después de desarrollar y enfocarse en detectar defectos; el ágil prueba continuamente, promueve la colaboración y prioriza la prevención y la retroalimentación.

9.2. Manifiesto del Testing Ágil

El Manifiesto expresa cinco preferencias:

Se valora más...	...que...
Probar durante el desarrollo	Probar únicamente al final.
Prevenir defectos	Encontrar defectos.
Comprender qué se está probando	Verificar solamente funcionalidades.
Construir un mejor sistema	Limitarse a romper el sistema.

Se valora más...	...que...
Responsabilidad compartida por la calidad	Responsabilidad exclusiva del tester.

La columna de la izquierda expresa las preferencias del Testing Ágil. No significa que los elementos de la derecha desaparezcan, sino que **se prioriza prevenir problemas y colaborar desde el inicio** por sobre limitarse a encontrarlos al terminar. El testing es una **actividad continua**, no una fase aislada.

9.3. Nueve principios

1. **Testing se mueve hacia adelante:** se piensa en pruebas desde los requerimientos y el diseño, para descubrir problemas temprano.
2. **Testing no es una fase:** se integra en todo el ciclo de desarrollo, no se reserva para el final.
3. **Todos hacen testing:** desarrolladores, testers y negocio colaboran en la calidad.
4. **Reducir la latencia del feedback:** obtener resultados rápidamente y corregir problemas cuanto antes.
5. **Las pruebas representan expectativas:** expresan qué debería hacer el producto y permiten verificarlo.
6. **Código limpio y defectos corregidos rápido:** evitar acumular problemas que dificulten cambios futuros.
7. **Reducir documentación innecesaria:** registrar información que aporte valor, sin convertirla en una carga.
8. **Las pruebas son parte del Done:** una funcionalidad no está terminada si no cumple sus condiciones de calidad.
9. **Desarrollo guiado por pruebas:** utilizar pruebas para orientar la implementación, en lugar de probar solamente al final.

9.4. Seis prácticas

1. **Pruebas unitarias y de integración automatizadas:** verifican con frecuencia componentes e interacciones y aportan feedback rápido.
2. **TDD:** escribir la prueba antes del código e implementar con el ciclo **Red → Green → Refactor**.

3. **Regresión automatizada de sistema:** repetir pruebas para detectar si los cambios afectan funcionalidades que antes funcionaban.
4. **ATDD:** acordar pruebas y criterios de aceptación desde las expectativas del negocio antes de implementar.
5. **Pruebas exploratorias:** investigar el sistema mientras se aprende y diseñan nuevos casos de prueba.
6. **Control de versiones de las pruebas junto con el código:** mantener las pruebas consistentes y trazables en cada versión.

TDD y ATDD están desarrollados con mayor detalle en **Unidad 3**.

9.5. Rol del tester

- **Con Product Owner:** clarificar historias y criterios de aceptación.
- **Con desarrolladores:** diseñar y automatizar pruebas; detectar defectos temprano.
- **Con todo el equipo:** definir Done, preparar ambientes, validar historias.

El tester **no espera un producto terminado**: participa desde las historias de usuario, ayuda a identificar riesgos y colabora con desarrollo y negocio. **Su rol no desaparece en Agile**; cambia su forma de trabajar. La calidad pertenece a todo el equipo, aunque el tester aporte conocimientos especializados.

9.6. Cuadrantes del Testing Ágil

Cuadrante	Orientación	Ejemplos
Q1	Tecnología; **guía el desarrollo**.	Unitarias/componentes.
Q2	Negocio; **guía el desarrollo**.	Criterios de aceptación, ejemplos y pruebas funcionales.
Q3	Negocio; **critica el producto**.	Exploratorias, usabilidad, aceptación.
Q4	Tecnología; **critica el producto**.	Carga, seguridad, rendimiento, recuperación.

Estos cuatro cuadrantes cruzan dos dimensiones: pruebas que **guían el desarrollo** o **evalúan críticamente el producto**, y pruebas orientadas al

negocio o a la **tecnología**.

Q1 + Q2 guían; Q3 + Q4 critican. Q1 + Q4 tecnología; Q2 + Q3 negocio.

9.7. Pirámide tradicional vs. ágil

La PPT compara una pirámide **tradicional invertida**, con muchas pruebas de interfaz, frente a una **ágil normal**, con una base amplia de pruebas unitarias.

attachment:19ba968c-fa4e-4766-b92a-9863152e6d85:piramide_testing_tradicional_vs_agil.html

Distribución ilustrativa	Tradicional	Ágil
80–90 %	GUI / End-to-End	**Unitarias**
5–15 %	Integración	Aceptación
1–5 %	Unitarias	GUI

Las pruebas unitarias suelen ser **más rápidas y menos costosas de mantener**; las GUI siguen siendo necesarias, pero en general cuestan más. Estos porcentajes corresponden al **esquema de la cátedra**, no a reglas universales.

9.8. Beneficios de automatizar

La presentación identifica seis beneficios:

1. **Rápida ejecución:** permite repetir casos de prueba muchas veces.
2. **Reusabilidad:** los casos automatizados pueden volver a ejecutarse.
3. **Mayor cobertura:** es posible comprobar más escenarios con frecuencia.
4. **Mayor precisión:** reduce errores asociados a la ejecución manual repetitiva.
5. **Mayor alcance:** facilita comprobar combinaciones o ambientes adicionales.
6. **Reducción de costos:** especialmente en pruebas de regresión frecuentes.

La automatización **no reemplaza el criterio humano ni las pruebas exploratorias**.

9.9. Desafío cultural

Incorporar testing ágil requiere un **cambio cultural** en toda la organización: colaboración entre roles, aceptación del feedback frecuente, gestión del cambio y expectativas compartidas. **No alcanza con automatizar pruebas**; el equipo debe incorporar la calidad como parte de cada entrega.

9.10. Lo principal para el parcial

Para el parcial: nueve principios, seis prácticas, **cuadrantes Q1–Q4**, rol del tester y **comparación de las dos pirámides**.